could catch you off guard: even if you call another_method after using, the call to my_method itself happens *before* using—so it calls the original, unrefined version of the method.

Some people find the result above counterintuitive. The lesson here is to double-check your method calls when you use *Refinements (36)*. Also keep in mind that Refinements are still an evolving feature—so much so that Ruby 2.0 issues a scary warning when your program uses Refinements for the first time:

```
warning: Refinements are experimental, and the
behavior may change in future versions of Ruby!
```

This warning has been removed in Ruby 2.1, but there are still a few corner cases where Refinements might not behave as you expect—and some of those corner cases might change in future Rubies. For example, you can call refine in a regular module, but you cannot call it in a class, even if a class is itself a module. Also, metaprogramming methods such as methods and ancestors ignore Refinements altogether. Behaviors such as these have sound technical justifications, but they could trip you up nonetheless. Refinements have the potential to eliminate dangerous Monkeypatches, but it will take some time for the Ruby community to understand how to use them best.

You're still considering the power and responsibility of using Refinements when Bill decides to throw a quiz at you.

Quiz: Tangle of Modules

Where you untangle a twisted yarn of modules, classes, and objects.

You can finally go back to the problem that prompted Bill to launch into his discussion on method lookup, self, and Refinements. You've had trouble making sense of a complicated arrangement of classes and modules. Here's the confusing part:

object_model/tangle.rb

```
module Printable
  def print
    # ...
  end

  def prepare_cover
    # ...
  end
end
```